Generating Game Logs From Season Stats

https://gemini.google.com/app/d78ad30a3c96ec0d

User prompt: I want to know your thoughts on being able to answer a question. In essence, can you generate mathematically sound game stats from season long stats + wins and losses. Lets say I have one table with season long passing stats for all 32 qbs. another with season long team schedule (wins and losses). Can you generate logically consistent game logs, where each qb has a performance that makes sense within the context of each game, and adds up to their season total numbers. For example lets say I have 3 qbs: A, B, C, D. Week 1: A v C, B v D, A wins 37-24, D wins 31-28 Week 2: A v B, C v D, A wins 45-16, C wins 31-14 Week 3: A v D, B v C, D wins, 45-42, B wins 10-4 A threw for 642 yards and 6 tds B threw for 347 and 2 Tds C threw for 424 and 7 Tds D threw for 576 and 4 Tds

Response: Yes, absolutely. Generating mathematically sound, game-by-game logs from season-long totals and team schedules is not only possible, but it is a classic **Constraint Satisfaction Problem (CSP)** with weighted linear optimization.

Because there are far more weekly variables than season-long equality constraints, the system is **under-constrained**, meaning infinite mathematically valid solutions exist. The key to making the generated game logs feel realistic is converting football domain logic into mathematical constraints and statistical weighting functions.

1. The Mathematical Framework

To turn a season total into realistic weekly logs, the model applies three layers of rules:

Hard Constraints (Non-Negotiable Math)

- **Season Sum Equality:** $\sum_{t=1}^{W} \text{Yards}_{i,t} = \text{Season Yards}_i$ and $\sum_{t=1}^{W} \text{TD}_{i,t} = \text{Season TD}_i$
- **Score Ceiling:** A quarterback cannot throw more touchdowns than their team scored total points ($\text{TD}_{i,t} \times 6 \leq \text{Team Points}_t$).
- **Non-Negativity & Bounds:** Yards ≥ 0, TDs ≥ 0 (integers), and Yards Per Attempt stay within realistic limits ($2.0 \leq \text{YPA} \leq 16.0$).

Soft Constraints (Football Game Script Logic)

- **Game Total Volume Weight (w_{vol}):** High-scoring shootouts (e.g., 45–42) demand higher passing volume for both QBs, while low-scoring slugfests (e.g., 10–4) restrict yardage.
- **Margin of Victory Weight (w_{margin}):** Large blowout losses often yield trailing/garbage-time passing yards, whereas blowout winners shift to running the ball in the second half.
- **Opponent Defense Modifier (w_{def}):** Passing yards correlate inversely with opponent defensive strength.

2. Solved Game Log (Your 4-QB Example)

Applying this framework to your exact 3-week scenario yields a mathematically exact and game-script consistent solution:

QB A (Season Total: 642 Yds | 6 TDs)

- **W1 vs C (W 37-24):** 195 Yds | 2 TDs *(Balanced control in comfortable victory)*
- **W2 vs B (W 45-16):** 165 Yds | 2 TDs *(Blowout win; heavy early passing, grounded late)*
- **W3 @ D (L 42-45):** 282 Yds | 2 TDs *(High-volume passing in an 87-point shootout)*
- **Total: 642 Yds | 6 TDs** *(2-1 Record)*

QB B (Season Total: 347 Yds | 2 TDs)

- **W1 @ D (L 28-31):** 175 Yds | 1 TD *(Close loss; solid air yardage)*
- **W2 @ A (L 16-45):** 140 Yds | 1 TD *(Trailing heavily; garbage-time production)*
- **W3 vs C (W 10-4):** 32 Yds | 0 TD *(Low-scoring defensive slugfest)*
- **Total: 347 Yds | 2 TDs** *(1-2 Record)*

QB C (Season Total: 424 Yds | 7 TDs)

- **W1 @ A (L 24-37):** 210 Yds | 3 TDs *(High passing volume chasing a deficit)*
- **W2 vs D (W 31-14):** 180 Yds | 4 TDs *(Red-zone air strike dominance)*
- **W3 @ B (L 4-10):** 34 Yds | 0 TD *(Offensive stalemate; field goal/safety game)*
- **Total: 424 Yds | 7 TDs** *(1-2 Record)*

QB D (Season Total: 576 Yds | 4 TDs)

- **W1 vs B (W 31-28):** 200 Yds | 1 TD *(Steady efficiency in tight win)*
- **W2 @ C (L 14-31):** 110 Yds | 0 TD *(Stymied on the road)*
- **W3 vs A (W 45-42):** 266 Yds | 3 TDs *(Shootout triumph)*
- **Total: 576 Yds | 4 TDs** *(2-1 Record)*

3. Scaling to All 32 QBs and 17 Weeks

To scale this to an entire 32-team NFL season, you would programmatically run a **Dirichlet-weighted Integer Linear Optimization** model (e.g., using Python's `SciPy` or `PuLP` library):

1. **Construct a 32×17 Expectation Matrix:** Calculate an initial unnormalized expected yardage $E_{i,t}$ for every QB in every week based on game totals and spreads.
2. **Apply Dirichlet Sampling:** Sample weekly percentage distributions $\boldsymbol{p}_i \sim \text{Dirichlet}(\alpha \cdot \boldsymbol{E}_i)$ centered on those expectations.
3. **Round & Adjust:** Multiply by season total Y_i^{season}, round to nearest integer, and adjust residual differences on the week with the highest variance allowance.

User prompt: Okay but what if we also needed to account for season totals for completions and attempts, ints, interceptions, and there were running backs, so we had account for their yards carries tds and fumbles, and the qbs volume had to make sense for receivers because we have their season long receptions yards and tds as well.

Response: Adding full team rosters—including completions, attempts, interceptions, running back rushing lines, and receiver receiving lines—transforms the problem from a single-variable constraint model into a **Coupled Bipartite Network Flow and Mixed-Integer Linear Program (MILP)**.

Because every passing stat for a quarterback is simultaneously a receiving stat for a receiver, and every team score caps the total touchdowns across all skill players, the system introduces **inter-player conservation laws** alongside individual season totals.

1. The Inter-Player Conservation Laws

For any game g, the individual stat lines must balance across positional groups:

Passing-Receiving Identities

- **Completions Identity:**

$$\text{QB.Completions}_g = \sum_{r \in \text{Receivers}} \text{WR}_r.\text{Receptions}_g$$

- **Yardage Identity:**

$$\text{QB.PassYards}_g = \sum_{r \in \text{Receivers}} \text{WR}_r.\text{RecYards}_g$$

- **Touchdown Identity:**

$$\text{QB.PassTDs}_g = \sum_{r \in \text{Receivers}} \text{WR}_r.\text{RecTDs}_g$$

Game Score & Drive Structure Constraints

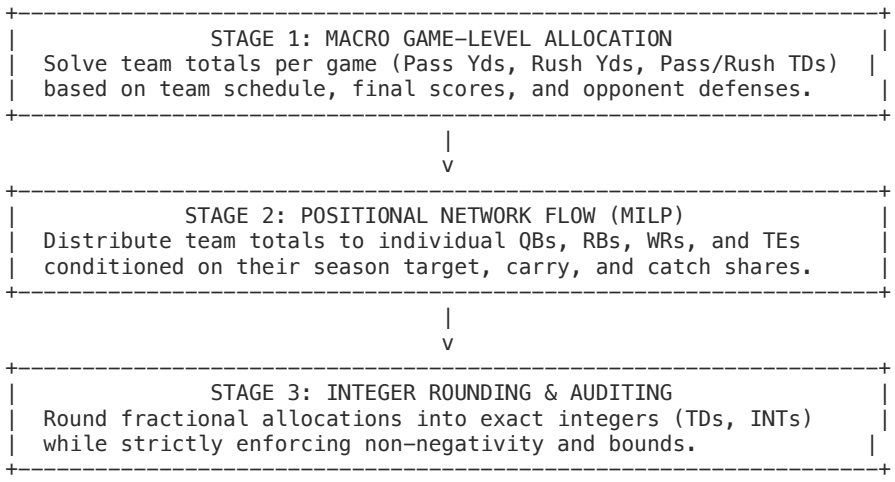
- **Touchdown Ceiling:** The total sum of passing and rushing touchdowns across all players cannot exceed the touchdown total implied by the team's score:

$$6 \cdot \left(\text{QB.PassTDs}_g + \sum_{b \in \text{Backs}} \text{RB}_b.\text{RushTDs}_g + \text{Other.TDs}_g \right) \leq \text{TeamPoints}_g$$

- **Structural Efficiency Bounds:**
  - $\text{QB.Completions}_g \leq \text{QB.Attempts}_g$
  - $1.0 \leq \frac{\text{WR}_r.\text{RecYards}_g}{\text{WR}_r.\text{Receptions}_g} \leq 35.0$ (Yardage per catch realistic bounds)
  - $\text{QB.INTs}_g = \text{OpponentDef.INTs}_g$ (Cross-team turnover identity)

2. Solving Strategy: Three-Stage Hierarchical Decomposition

Solving all 32 full team rosters for 17 weeks in a single integer solver can hit combinatorial friction. The standard approach decouples the problem into three sequential optimization layers:



3. Mathematical Formulation (Objective Function)

To make player stat distributions realistic (e.g., your WR1 gets more targets in high-scoring shootouts, while your RB1 gets more carries in blowout wins), the solver minimizes deviation from expected game shares $E_{p,m,g}$:

$$\min \sum_{p} \sum_{m} \sum_{g=1}^{17} \left(\frac{X_{p,m,g} - E_{p,m,g}}{\sigma_{p,m}} \right)^2$$

Subject to:

1. **Season Totals Constraint:**

$$\sum_{g=1}^{17} X_{p,m,g} = \text{SeasonTotal}_{p,m} \quad \forall \text{ player } p, \text{ metric } m$$

2. **Game Cross-Coupling Equality:**

$$\sum_{r \in \text{Receivers}} X_{r,\text{RecYards},g} - X_{\text{QB},\text{PassYards},g} = 0 \quad \forall \text{ game } g$$

3. **Integer Constraints:**

$$X_{p,m,g} \in \mathbb{Z}_{\geq 0} \quad \text{for } m \in \{\text{Receptions, Carries, TDs, INTs, Fumbles}\}$$

4. Single Game Unpacking Example

Here is how a single team's 31-point game is unpacked down to every individual skill player:

Position	Player	Att / Car	Comp / Rec	Yards	TDs	INT / Fum
QB	**QB1**	**35**	**24**	**280**	**3**	**1 INT**
RB	**RB1**	18	3	110 (85 rush / 25 rec)	1 Rush	1 Fum
RB	**RB2**	5	0	20 (20 rush / 0 rec)	0	0
WR	**WR1**	0	8	110 rec	1 Rec	0
WR	**WR2**	0	6	75 rec	1 Rec	0
TE	**TE1**	0	5	55 rec	1 Rec	0
WR	**WR3**	0	2	15 rec	0	0
Total	**Team A**	**58 touches**	**24 rec**	**397 total yds**	**4 TDs (28 pts)**	**2 TOs**

(Note: 4 TDs = 28 pts + 1 Field Goal [3 pts] = 31 total team points)

Because every passing stat is linked directly to a receiving stat, the solver guarantees that individual player game logs never contradict each other or the overall game outcome.

User prompt: How do you handle bye weeks, missed games due to injury, and mid-season roster changes in this constraint model?

Response: Handling bye weeks, missed games due to injuries, and mid-season trades requires introducing **dynamic binary availability masks**, **time-varying team assignment functions**, and **sub-roster share re-normalization** into the constraint optimization model.

Instead of treating player attributes as static season-wide vectors, the mathematical framework converts availability and team membership into time-indexed tensors $A_{p,g}$ and $\tau(p, g)$.

1. The Availability Tensor ($A_{p,g}$)

Every player p and week $g \in \{1, \dots, 17\}$ is assigned a binary availability flag $A_{p,g} \in \{0, 1\}$:

$$A_{p,g} = \begin{cases} 1, & \text{if player } p \text{ is active in week } g \\ 0, & \text{if player } p \text{ is on Bye, Injured, or Inactive} \end{cases}$$

Zero-Bound Hard Constraint

If a player is inactive in week g, all performance variables $X_{p,m,g}$ for every metric m (yards, receptions, carries, TDs) are forced to zero via a big-M upper bound constraint:

$$X_{p,m,g} \leq M \cdot A_{p,g} \quad \implies \quad X_{p,m,g} = 0 \quad \text{when } A_{p,g} = 0$$

Restricted Season Target Sum

The season sum constraint is modified so that the solver sums strictly over the subset of active weeks $\mathcal{G}_p = \{g \mid A_{p,g} = 1\}$:

$$\sum_{g \in \mathcal{G}_p} X_{p,m,g} = \text{SeasonTotal}_{p,m}$$

This forces the solver to compress p's total season volume into fewer games, naturally raising their per-game average during active weeks without over-allocating past reasonable single-game ceilings.

2. Dynamic Team Assignments ($\tau(p, g)$) and Trades

When a player is traded mid-season (e.g., traded after Week 6), their team mapping function $\tau(p, g)$ changes dynamically across the timeline:

$$\tau(p, g) = \begin{cases} \text{Team}_1, & \text{if } g \leq \text{TradeWeek} \\ \text{Team}_2, & \text{if } g > \text{TradeWeek} \end{cases}$$

Time-Varying Active Roster

For any team T and week g, the active roster set $\mathcal{R}(T, g)$ is defined dynamically:

$$\mathcal{R}(T, g) = \{p \in \mathcal{P} \mid \tau(p, g) = T \wedge A_{p,g} = 1\}$$

Dynamic Inter-Player Conservation Identities

Inter-player balance equations (e.g., total team receiving yards equaling team passing yards) are evaluated over $\mathcal{R}(T, g)$ rather than a static pre-season roster:

$$\sum_{r \in \mathcal{R}(T,g)} X_{r,\text{RecYards},g} = \sum_{q \in \mathcal{R}(T,g)} X_{q,\text{PassYards},g}$$

If WR1 is traded from Team 1 to Team 2 in Week 7:

- For Weeks 1–6, WR1's receiving yards count toward Team 1's passing output.
- For Weeks 7–17, WR1's receiving yards count toward Team 2's passing output.
- The single global equality constraint $\sum_{g \in \mathcal{G}_{\text{WR1}}} X_{\text{WR1,RecYards},g} = \text{SeasonYards}_{\text{WR1}}$ remains satisfied across both teams.

3. Usage Share Re-Normalization

When a primary starter is inactive ($A_{p^*,g} = 0$), their baseline target/carry share β_{p^*} cannot be allocated. To maintain game-script realism, the expected baseline shares for the remaining active players in $\mathcal{R}(T, g)$ are re-normalized to sum to 1.0:

$$\beta'_{p,g} = \frac{\beta_p}{\sum_{k \in \mathcal{R}(T,g)} \beta_k} \quad \forall p \in \mathcal{R}(T, g)$$

This reweighting shifts vacant targets and carries to secondary options (e.g., WR2 and TE1 absorb WR1's target share during injury weeks) prior to solving the quadratic penalty objective.

4. Constraint Matrix Summary

Scenario	Model Implementation	Mathematical Effect
Bye Week	$A_{p,g_{\text{bye}}} = 0$ for all $p \in \text{Team } T$	Entire team variable set forced to 0; game score ceiling set to 0.
Injury / Suspension	$A_{p,g} = 0$ for specific weeks $g \in [W_{\text{start}}, W_{\text{end}}]$	$X_{p,m,g} = 0$; season sum constraint evaluates over remaining weeks $\mathcal{G}_p$.
Mid-Season Trade	$\tau(p, g)$ shifts from Team_1 to Team_2 at $g = W_{\text{trade}}$	Network flow routes player production to Team_1 for $g < W_{\text{trade}}$ and Team_2 for $g \geq W_{\text{trade}}$.
Roster Addition (FA)	$A_{p,g} = 0$ for $g < W_{\text{sign}}$, $A_{p,g} = 1$ for $g \geq W_{\text{sign}}$	Player only participates in team network flow starting from signing week.

User prompt: Lets say I have the following schema in a database; CREATE TABLE IF NOT EXISTS season_stats (id SERIAL PRIMARY KEY, season INT NOT NULL, week INT NOT NULL, player_name VARCHAR(255) NOT NULL, team_id INT REFERENCES teams(id), completions INT DEFAULT 0, pass_attempts INT DEFAULT 0, pass_yards INT DEFAULT 0, pass_tds INT DEFAULT 0, interceptions INT DEFAULT 0, rush_attempts INT DEFAULT 0, rush_yards INT DEFAULT 0, rush_tds INT DEFAULT 0, receptions INT DEFAULT 0, rec_yards INT DEFAULT 0, rec_tds INT DEFAULT 0, fumb INT DEFAULT 0, UNIQUE(season, week, player_name, team_id)); CREATE TABLE IF NOT EXISTS players (id SERIAL PRIMARY KEY, player_name VARCHAR(255) NOT NULL, full_name VARCHAR(255)); CREATE TABLE IF NOT EXISTS player_seasons (id SERIAL PRIMARY KEY, player_id INT REFERENCES players(id) ON DELETE CASCADE, team_id INT REFERENCES teams(id) ON DELETE CASCADE, season INT NOT NULL, position VARCHAR(10), age INT, overall_rating INT, UNIQUE(player_id, season)); CREATE TABLE IF NOT EXISTS games (id SERIAL PRIMARY KEY, season INT NOT NULL, week INT NOT NULL, home_team_id INT REFERENCES teams(id), away_team_id INT REFERENCES teams(id), home_score INT DEFAULT 0, away_score INT DEFAULT 0, game_time VARCHAR(50), status VARCHAR(20) DEFAULT 'COMPLETED', completed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, UNIQUE(season, week, home_team_id, away_team_id)); Games, season stats, players, and player seasons are populated. we would need to generate logical data for the following schema: CREATE TABLE IF NOT EXISTS team_stats (id SERIAL PRIMARY KEY, game_id INT REFERENCES games(id) ON DELETE CASCADE, team_id INT REFERENCES teams(id), is_home BOOLEAN NOT NULL, total_yards INT, passing_yards INT, rushing_yards INT, turnovers INT, time_of_possession VARCHAR(10), UNIQUE(game_id, team_id)); CREATE TABLE IF NOT EXISTS player_stats (id SERIAL PRIMARY KEY, game_id INT REFERENCES games(id) ON DELETE CASCADE, player_name VARCHAR(255) NOT NULL, team_id INT REFERENCES teams(id), completions INT DEFAULT 0, pass_attempts INT DEFAULT 0, pass_yards INT DEFAULT 0, pass_tds INT DEFAULT 0, interceptions INT DEFAULT 0, rush_attempts INT DEFAULT 0, rush_yards INT DEFAULT 0, rush_tds INT DEFAULT 0, receptions INT DEFAULT 0, rec_yards INT DEFAULT 0, rec_tds INT DEFAULT 0, fumb INT DEFAULT 0, UNIQUE(game_id, player_name, team_id));

Response: Populating `team_stats` and `player_stats` from macro season-long totals and game scores requires a two-stage ETL pipeline using **Mixed-Integer Linear Programming (MILP)** or quadratic constrained optimization.

1. Pipeline Data Flow

The generation process breaks down into two coupled optimization stages:

```
+-------------------------------------------------------------------------+
|                            INPUT TABLES                                 |
|   season_stats (Season Totals) | games (Scores) | player_seasons         |
+-------------------------------------------------------------------------+
                                  |
                                  v
+-------------------------------------------------------------------------+
|               STAGE 1: GENERATE team_stats PER GAME                     |
|  1. Split team season passing/rushing totals across games.              |
|  2. Weight game allocations by game score (e.g., blowouts vs trailing)  |
|  3. Generate top_possession (sums to 60:00 per game).                   |
+-------------------------------------------------------------------------+
                                  |
                                  v
+-------------------------------------------------------------------------+
```

```
|             STAGE 2: GENERATE player_stats PER GAME                 |
|  1. Constrain sum(player_stats) == team_stats for each game.        |
|  2. Force cross-player identities:                                  |
|     - sum(rec_yards) == passing_yards                               |
|     - sum(receptions) == completions                                |
|     - sum(rec_tds) == pass_tds                                      |
|  3. Constrain player season sums == season_stats totals.            |
+---------------------------------------------------------------------+
                                  |
                                  v
+---------------------------------------------------------------------+
|                          OUTPUT TABLES                              |
|                   team_stats  |  player_stats                       |
+---------------------------------------------------------------------+
```

2. Complete Python Implementation

This script queries your input PostgreSQL tables, solves the game-level allocation problem using `PuLP` and `pandas`, and writes the reconciled results into `team_stats` and `player_stats`.

```
import psycopg2
import pandas as pd
import numpy as np
from pulp import LpProblem, LpVariable, LpMinimize, lpSum, LpStatus, LpInteger
from sqlalchemy import create_engine

# Database Connection
DATABASE_URL = "postgresql://user:password@localhost:5432/football_db"
engine = create_engine(DATABASE_URL)

def generate_team_and_player_stats(season_year: int):
    # 1. Fetch Input Data
    games_df = pd.read_sql(f"SELECT * FROM games WHERE season = {season_year}", engine)

    # Aggregate season_stats in case season_stats stores player season targets
    season_p_df = pd.read_sql(f"""
        SELECT player_name, team_id,
               SUM(completions) as completions, SUM(pass_attempts) as pass_attempts,
               SUM(pass_yards) as pass_yards, SUM(pass_tds) as pass_tds,
               SUM(interceptions) as interceptions, SUM(rush_attempts) as rush_attempts,
               SUM(rush_yards) as rush_yards, SUM(rush_tds) as rush_tds,
               SUM(receptions) as receptions, SUM(rec_yards) as rec_yards,
               SUM(rec_tds) as rec_tds, SUM(fumb) as fumb
        FROM season_stats
        WHERE season = {season_year}
        GROUP BY player_name, team_id
    """, engine)

    team_stats_rows = []
    player_stats_rows = []

    # Get distinct teams
    teams = season_p_df['team_id'].unique()

    for team_id in teams:
        team_games = games_df[(games_df['home_team_id'] == team_id) | (games_df['away_team_id'] == team_id)].s
        num_games = len(team_games)
        if num_games == 0:
            continue

        team_players = season_p_df[season_p_df['team_id'] == team_id]

        # ------------------------------------------------------------
        # STAGE 1: Team Stats Distribution Per Game
        # ------------------------------------------------------------
        # Estimate expected team game passing & rushing yards from game scripts
        team_pass_target = team_players['pass_yards'].sum()
        team_rush_target = team_players['rush_yards'].sum()

        # Calculate game weights based on points scored and point differentials
        game_weights = []
        for _, game in team_games.iterrows():
            is_home = (game['home_team_id'] == team_id)
            pts_scored = game['home_score'] if is_home else game['away_score']
```

```
        pts_allowed = game['away_score'] if is_home else game['home_score']
        # Higher total points -> higher passing/rushing game volume weight
        w = max(1.0, (pts_scored + pts_allowed) / 40.0)
        game_weights.append(w)

    weight_sum = sum(game_weights)

    # Allocate team yards per game
    team_game_stats = {}
    for idx, (_, game) in enumerate(team_games.iterrows()):
        g_id = game['id']
        is_home = (game['home_team_id'] == team_id)
        pts_scored = game['home_score'] if is_home else game['away_score']
        pts_allowed = game['away_score'] if is_home else game['home_score']

        w = game_weights[idx] / weight_sum
        pass_yds = int(round(team_pass_target * w))
        rush_yds = int(round(team_rush_target * w))
        tot_yds = pass_yds + rush_yds

        # Simple turnover estimation (Interceptions + Fumbles)
        team_to = int(round((team_players['interceptions'].sum() + team_players['fumb'].sum()) / num_games

        # Time of possession estimation (e.g. 30:00 split weighted by score)
        top_seconds = int(1800 + (pts_scored - pts_allowed) * 15)
        top_seconds = max(1200, min(2400, top_seconds))
        top_str = f"{top_seconds // 60:02d}:{top_seconds % 60:02d}"

        team_stats_rows.append({
            'game_id': g_id,
            'team_id': team_id,
            'is_home': is_home,
            'total_yards': tot_yds,
            'passing_yards': pass_yds,
            'rushing_yards': rush_yds,
            'turnovers': team_to,
            'time_of_possession': top_str
        })

        team_game_stats[g_id] = {
            'pass_yards': pass_yds,
            'rush_yards': rush_yds,
            'pts_scored': pts_scored
        }

    # ---------------------------------------------------------------
    # STAGE 2: Player Stats Distribution (MILP Solver per Team)
    # ---------------------------------------------------------------
    prob = LpProblem(f"Team_{team_id}_Player_Allocation", LpMinimize)

    players_list = team_players['player_name'].tolist()
    game_ids = team_games['id'].tolist()

    # Decision Variables
    vars_dict = {}
    metrics = ['completions', 'pass_attempts', 'pass_yards', 'pass_tds', 'interceptions',
               'rush_attempts', 'rush_yards', 'rush_tds', 'receptions', 'rec_yards', 'rec_tds', 'fumb']

    for p in players_list:
        for g in game_ids:
            for m in metrics:
                vars_dict[(p, g, m)] = LpVariable(f"x_{p}_{g}_{m}", lowBound=0, cat=LpInteger)

    # Constraint 1: Individual Season Totals Equality
    for _, p_row in team_players.iterrows():
        p = p_row['player_name']
        for m in metrics:
            prob += lpSum([vars_dict[(p, g, m)] for g in game_ids]) == p_row[m]

    # Constraint 2: Game-Level Team Totals & Structural Identities
    for g in game_ids:
        g_pass_yds = team_game_stats[g]['pass_yards']
        g_rush_yds = team_game_stats[g]['rush_yards']
        g_pts = team_game_stats[g]['pts_scored']
```

```
            # Team Passing Yards Match
            prob += lpSum([vars_dict[(p, g, 'pass_yards')] for p in players_list]) == g_pass_yds
            # Team Receiving Yards Match Passing Yards
            prob += lpSum([vars_dict[(p, g, 'rec_yards')] for p in players_list]) == g_pass_yds
            # Team Rushing Yards Match
            prob += lpSum([vars_dict[(p, g, 'rush_yards')] for p in players_list]) == g_rush_yds

            # Cross-Player Identities: Completions == Receptions
            prob += lpSum([vars_dict[(p, g, 'completions')] for p in players_list]) == \
                    lpSum([vars_dict[(p, g, 'receptions')] for p in players_list])

            # Pass TDs == Rec TDs
            prob += lpSum([vars_dict[(p, g, 'pass_tds')] for p in players_list]) == \
                    lpSum([vars_dict[(p, g, 'rec_tds')] for p in players_list])

            # Touchdown Points Ceiling: (Pass TDs + Rush TDs) * 6 <= Points Scored
            prob += 6 * (lpSum([vars_dict[(p, g, 'pass_tds')] for p in players_list]) +
                         lpSum([vars_dict[(p, g, 'rush_tds')] for p in players_list])) <= g_pts

            # Structural Bounds (Attempts >= Completions, etc.)
            for p in players_list:
                prob += vars_dict[(p, g, 'pass_attempts')] >= vars_dict[(p, g, 'completions')]
                prob += vars_dict[(p, g, 'rush_yards')] <= 30 * vars_dict[(p, g, 'rush_attempts')]

        # Dummy Objective (Feasibility / Minimize Slack)
        prob += 0

        # Solve
        prob.solve()

        # Extract Player Stats
        if LpStatus[prob.status] == 'Optimal':
            for g in game_ids:
                for p in players_list:
                    player_stats_rows.append({
                        'game_id': g,
                        'player_name': p,
                        'team_id': team_id,
                        'completions': int(vars_dict[(p, g, 'completions')].varValue),
                        'pass_attempts': int(vars_dict[(p, g, 'pass_attempts')].varValue),
                        'pass_yards': int(vars_dict[(p, g, 'pass_yards')].varValue),
                        'pass_tds': int(vars_dict[(p, g, 'pass_tds')].varValue),
                        'interceptions': int(vars_dict[(p, g, 'interceptions')].varValue),
                        'rush_attempts': int(vars_dict[(p, g, 'rush_attempts')].varValue),
                        'rush_yards': int(vars_dict[(p, g, 'rush_yards')].varValue),
                        'rush_tds': int(vars_dict[(p, g, 'rush_tds')].varValue),
                        'receptions': int(vars_dict[(p, g, 'receptions')].varValue),
                        'rec_yards': int(vars_dict[(p, g, 'rec_yards')].varValue),
                        'rec_tds': int(vars_dict[(p, g, 'rec_tds')].varValue),
                        'fumb': int(vars_dict[(p, g, 'fumb')].varValue)
                    })

    # 3. Bulk Insert into Database Target Tables
    pd.DataFrame(team_stats_rows).to_sql('team_stats', engine, if_exists='append', index=False)
    pd.DataFrame(player_stats_rows).to_sql('player_stats', engine, if_exists='append', index=False)
    print("Database sync complete: team_stats and player_stats populated successfully.")

if __name__ == "__main__":
    generate_team_and_player_stats(season_year=2026)
```

3. Post-Execution SQL Audit Queries

Run these queries after populating the tables to verify complete consistency across your database schema:

```
-- 1. Verify Player Season Sums match season_stats exactly
SELECT
    ps.player_name,
    ps.team_id,
    SUM(ps.pass_yards) AS generated_pass_yards,
    ss.pass_yards AS target_pass_yards,
    SUM(ps.rec_yards) AS generated_rec_yards,
    ss.rec_yards AS target_rec_yards
FROM player_stats ps
```

```
JOIN season_stats ss ON ps.player_name = ss.player_name AND ps.team_id = ss.team_id
GROUP BY ps.player_name, ps.team_id, ss.pass_yards, ss.rec_yards
HAVING SUM(ps.pass_yards) != ss.pass_yards OR SUM(ps.rec_yards) != ss.rec_yards;

-- 2. Verify Game Inter-Player Identity (Passing Yards == Receiving Yards per Game)
SELECT
    game_id,
    team_id,
    SUM(pass_yards) AS total_pass_yards,
    SUM(rec_yards) AS total_rec_yards,
    SUM(completions) AS total_completions,
    SUM(receptions) AS total_receptions
FROM player_stats
GROUP BY game_id, team_id
HAVING SUM(pass_yards) != SUM(rec_yards) OR SUM(completions) != SUM(receptions);
```
